`ob_start()` creates an output buffer; from this point on everything, we output with `print`, etc., will end up stored in here. `ob_get_clean()` returns the contents of the buffer, which we store in the `$buffered` variable. From the output of the program, we can see that the first three `print` statements were buffered because they appear after the `Buffer contents:` line. There will be more to come on output buffering in Chapter 11, "Programming for the Internet."

## Command Line Input

As stated above, the `register_argc_argv` directive controls whether or not command line arguments are made available to your PHP code. When enabled it provides a number of mechanisms you can use to access any data entered at the command line when the PHP binary is invoked. Here's a simple program – which we'll save in a file called `argv_argc.php` – that outputs the values of some special variables. PHP variables always start with the $ symbol followed by some kind of symbolic name; in this case, these are `$argc`, `$argv`, and `$_SERVER`. Each of these variables happens to be of a type called an *array*, which is a collection of scalar (single value) values (e.g., numbers, strings of characters, booleans). Each scalar value inside an array is an *element* of that array.

```
// argv_argc.php
var_dump($argc);
print_r($argv);
print_r($_SERVER['argv']);
```

The syntax `$_SERVER['argv']` is an example of *array access*: the square bracket notation tells PHP that you want to access one of the scalar values, and the bit inside the square bracket tells PHP which particular element you want to access. There is an entire chapter of this book devoted to arrays (called, in an act of stunning imagination on my part, Chapter 5, "Arrays"), and you are quite free to have a sneaky look at it right now if you like. Now let's run our code from the command line and supply some data for it to make use of:

```
$ php argv_argc.php some data
int(3)
```